

Notes on the Theory of Finite-Horizon MDPs

Mohammad Sadegh Talebi
University of Copenhagen
sadegh.talebi@di.ku.dk

Initial draft: June 15, 2022

This version: March 10, 2026

In this note, we study finite-horizon Markov Decision Processes (or finite-horizon MDPs, for short). [1, 2] provide an elegant treatment of this subject.

Notations. We introduce notations that will be used throughout. Given a set A , $\Delta(A)$ denotes the simplex of probability distributions over A . $\mathbb{I}\{A\}$ denotes the indicator function of event A , namely it equals 1 if A occurs, else 0. $\mathbb{N}$ and $\mathbb{R}$ denotes the set of naturals and reals, respectively. Unless specified, $\|\cdot\|$ is used to denote a generic norm. We write $X \sim p$ to indicate that X is drawn randomly from distribution p . Finally, given a set A , $|A|$ denotes the cardinality of A , and a function $f : A \rightarrow \mathbb{R}^{|A|}$ could interchangeably be viewed as a $|A|$ -dimensional vector.

1 Finite-Horizon MDPs: Definition

Let $H \in \mathbb{N}$ be a given finite-horizon, and define $[H] := \{1, \dots, H\}$. A finite-horizon MDP M with horizon H is a tuple

$$M = \left(\mathcal{S}, \mathcal{A}, \{P_h, h \in [H]\}, \{R_h, h \in [H]\}, H \right),$$

where:

- $\mathcal{S}$ denotes the state-space, which is the set of all possible states the agent may occupy. Moreover, $\mathcal{S}_{\text{term}} \subset \mathcal{S}$ denotes the set of *terminal* states, i.e., the collection of states the agent may be in at time step H .
- $\mathcal{A}$ denotes the action-space, which is the set of possible actions available to the agent at any state $s \in \mathcal{S} \subset \mathcal{S}_{\text{term}}$. We assume $\mathcal{A} = \emptyset$ for all $s \in \mathcal{S}_{\text{term}}$, namely the agent has no action to choose in a terminal state.
- $P_h : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ denotes the transition function such that $P_h(\cdot|s, a)$ is the probability distribution of the next-states when action $a \in \mathcal{A}$ in state $s \in \mathcal{S} \setminus \mathcal{S}_{\text{term}}$ and time step $h \in [H - 1]$ is selected; and
- $R_h : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ denotes the reward function such that $R_h(s, a)$ denotes the probability distribution of (possibly random) rewards obtained when choosing action $a \in \mathcal{A}$ in state $s \in \mathcal{S}$ and time step $h \in [H]$. Since $\mathcal{A} = \emptyset$ for all $s \in \mathcal{S}_{\text{term}}$, we write $R_H(s)$ to denote the reward distribution in $s \in \mathcal{S}_{\text{term}}$.

The state space $\mathcal{S}$ and action space $\mathcal{A}$ maybe finite, countably infinite, or even continuous. In this note, we restrict the attention on discrete MDPs, where both $\mathcal{S}$ and $\mathcal{A}$ are discrete (i.e., finite or countably infinite). In a discrete finite-horizon MDP M , the transition function $\{P_h\}$ is a collection of discrete probability distributions or probability vectors. In particular, $P_h(\cdot|s, a)$ denotes the probability distribution of next-states for $s \in \mathcal{S}$ and $a \in \mathcal{A}_s$, and in time step $h \in [H]$. Hence, $\sum_{x \in \mathcal{S}} P_h(x|s, a) = 1$ for all a and h . We may also treat $P_h(\cdot|s, a)$ as a probability vector of dimension $S := |\mathcal{S}|$.

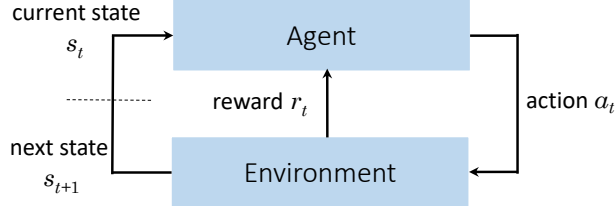


Figure 1: The interaction between the agent and the environment (MDP)

Remark 1 This definition can be readily extended to capture state-dependent action-spaces, namely where the set of actions differs in various states. In such cases, we write $\mathcal{A}_s$ to denote the set of actions available at state $s \in \mathcal{S}$. Unless stated otherwise, we assume $\mathcal{A}_s = \mathcal{A}$ for all $s \in \mathcal{S} \setminus \mathcal{S}_{\text{term}}$.

The Interaction between MDP and the Agent. Consider a time slotted system, where time is divided into slots (or periods) of identical lengths indexed by $t \in [H]$. The agent interacts with the MDP (environment) for H steps as follows. She starts in one initial state s_1 , which is determined by Nature, according to some distribution, which might be known or unknown to the agent. At each time period $t \in [H - 1]$, the agent is in state $s_t \in \mathcal{S} \setminus \mathcal{S}_{\text{term}}$. At the beginning of time period $t \in [H - 1]$, she chooses an action $a_t \in \mathcal{A}$ according to some (possibly randomized) action selection rule and executes it. Upon executing a_t in s_t , the agent receives a (possibly random) reward r_t from the environment, which is sampled from the reward distribution $R_t(s_t, a_t)$, namely, $r_t \sim R_t(s_t, a_t)$. The agent may receive r_t immediately, or later during the period: in one shot or multiple phases. However, we do assume that she will earn the entire r_t until the end of the period and before deciding a_{t+1} . Upon executing a_t in s_t , the environment generates a next-state s_{t+1} according to $P_t(\cdot | s_t, a_t)$, i.e., $s_{t+1} \sim P_t(\cdot | s_t, a_t)$. Then, once the new slot begins, the environment transits to s_{t+1} and a new round of decision step starts. This interaction is depicted in Figure 1. Finally, at time period $t = H$, the agent is in some terminal $s_H \in \mathcal{S}_{\text{term}}$, has no action to choose (i.e., $\mathcal{A} = \emptyset$), and receives a reward r_H distributed according to $R_H(s)$. This interaction yields a history h_t at each time $t \in [H]$:

$$h_t = (s_1, a_1, s_2, a_2, \dots, s_{t-1}, a_{t-1}, s_t).$$

The goal of the agent is to maximize the total discounted rewards collected during her interaction with the environment. More formally, she seeks to solve:

$$\max_{\text{strategies}} \mathbb{E} \left[\sum_{t=1}^H r_t \right]$$

for any choice of initial state, where the expectation is taken with respect to possible randomness in the initial states, the randomness in the received rewards and state transitions, and possible randomization in the action selection. Here, the maximization is done over all possible decision making (action selection) strategies. This defines a notion of optimality, which is called *the expected total reward optimality criterion*.

Note that the definition above indicates that a *non-stationary* MDP setting, where the reward and transition functions may vary over time. In fact, the solution procedures for finite-horizon MDPs allow us to consider such a generality.

Assumption 1 In the rest of this note, we assume deterministic reward functions. That is, we assume choosing a in s and time h yields $r_h = r_h(s, a) = R_h(s, a)$. Then, the optimization problem becomes

$$\max \mathbb{E} \left[\sum_{t=1}^{H-1} r_t(s_t, a_t) + r_H(s_H) \right]$$

However, we remark that the results in this note will hold for the case of random rewards if one replaces $r(s, a)$ with the mean of the reward distribution $R(s, a)$.

2 Policy and Value Function

2.1 Notions of Policy

As in discounted MDPs, the agent when interacting with a finite-horizon MDP chooses actions based on a **policy**, which prescribes what action to choose at various states and time steps, and might be:

- *Randomized or deterministic*: A randomized policy¹ outputs a probability distribution over actions whereas a deterministic policy outputs a single action.
- *History-dependent or Markov*: A history-dependent policy determines the action based on the history (hence, it is necessarily time-varying, or non-stationary), whereas a Markov one uses the current state only, yet might change over time.

Hence, we identify *four classes* of policies as follows. A **history-dependent randomized** policy is a mapping of the form $\pi : \mathcal{H} \rightarrow \Delta(\mathcal{A})$, where $\mathcal{H}$ is the set of all possible trajectories and $\Delta(\mathcal{A})$ denotes the set of probability distributions over $\mathcal{A}$. Here, π maps a history $h_t \in \mathcal{H}$ of observations up to time t (i.e., a sequence $(s_1, a_1, r_1, \dots, s_{t-1}, a_{t-1}, r_{t-1}, s_t)$) to a probability distribution over $\mathcal{A}$. Let Π^{HR} denote the set of all history-dependent randomized policies. (Obviously, Π^{HR} depends on the MDP of interest.) We write $a_t \sim \pi(h_t)$ to indicate that a_t is sampled from $\pi(h_t)$. Such a policy could be complicated to use in practice since it depends on the history and it prescribes choosing actions in a randomized manner. A **history-dependent deterministic policy** has the form $\pi : \mathcal{H} \rightarrow \mathcal{A}$. Hence, it prescribes $a_t = \pi(h_t)$. Let Π^{HD} denote the set of all history-dependent deterministic policies. A **Markov randomized policy** is a mapping $\pi : \mathcal{S} \times [H] \rightarrow \Delta(\mathcal{A})$, which uses the current state to choose an action, but the choice of the output action depends on time; hence $a_t \sim \pi(s_t, t)$. Let Π^{MR} denote the set of all Markov randomized policies. Finally, a **Markov deterministic policy** is a mapping $\pi : \mathcal{S} \times [H] \rightarrow \mathcal{A}$, and $a_t = \pi(s_t, t)$. Let Π^{MD} denote the set of all stationary deterministic policies.

We have $\Pi^{\text{MD}} \subset \Pi^{\text{MR}} \subset \Pi^{\text{HR}}$ and $\Pi^{\text{MD}} \subset \Pi^{\text{HD}} \subset \Pi^{\text{HR}}$. Table 1 summarizes the notions of policies above.

	Deterministic	Stochastic
Markov	$\pi : \mathcal{S} \times [H] \rightarrow \mathcal{A}$	$\pi : \mathcal{S} \times [H] \rightarrow \Delta(\mathcal{A})$
History-dependent	$\pi : \mathcal{H}_t \rightarrow \mathcal{A}$	$\pi : \mathcal{H}_t \rightarrow \Delta(\mathcal{A})$

Table 1: Various notions of policies in a finite-horizon MDP

2.2 Value Function

Let $\pi \in \Pi^{\text{HR}}$ in M . Informally, we can define the value of π in state s as the expected total reward achieved under π during H steps of interactions starting from s . Then, similarly to the case of discounted MDPs, we can cast the optimization in the MDP as finding a policy with the maximal value. A first step in doing so is to develop a method to compute the value of a fixed policy π . As we will shortly see, this can be done using a wise inductive procedure that proceeds in a backward way: For a given π , first compute the expected reward collected (under π) in the last round H . Having computed that, find the expected reward obtained in the last two rounds ($H - 1$ and H): This is simply done by looking at the immediate reward in $H - 1$, added to the one from H . Following this *backward induction* for $H - 1$ rounds yields the expected total reward obtained in rounds $2, \dots, H$. Now the H -step expected reward is simply the reward obtained in the first round added to the one obtained in round $2, \dots, H$ (which is already computed).

This procedure will be formalized as a theorem below. But the informal description above shall provide some insight into why we may define a notion of value function which depends on both (starting

¹Some texts call it stochastic policy.

state and the time, as follows. Formally, we define the **state value function** of π (or for short, the **value (function)** π) in M as a mapping $V_M^\pi : \mathcal{S} \times [H] \rightarrow \mathbb{R}$, such that for $s \in \mathcal{S}$ and $h \in [H]$, $V_M^\pi(s, h)$ is defined as the expected sum of rewards obtained by always following π , when starting from s at time h . Formally:

$$V_M^\pi(s, h) := \mathbb{E}^\pi \left[\sum_{t=h}^{H-1} r_t(s_t, a_t) + r_H(s_H) \middle| s_h = s \right],$$

where $\mathbb{E}^\pi$ indicates that the expectation is taken over trajectories generated by π . Here, the subscript M indicates that the underlying MDP is M . We write $V^\pi := V_M^\pi$ whenever it is clear from the context that the underlying MDP is M .

Intuitively, $V^\pi(s, h)$ measures the sum of $H - h + 1$ future rewards collected (in expectation) in rounds $h, h + 1, \dots, H$ under π when the agent is in state s at time h (i.e., $s_h = s$). Hence, the following trivial bound holds: $V^\pi(s, h) \leq (H - h + 1) \max_{t,s,a} |r_t(s, a)|$. Further, we remark that we are interested in H -step reward collected under a policy π , so the ultimate quantity of interest is $V^\pi(s, 1)$ that measure the rewards collected in the entire interaction. As you will see shortly, $V^\pi(s, h)$ for $h \neq 1$ is introduced as a proxy to find $V^\pi(s, 1)$.

2.3 Optimal Policy and Values

Solving a finite-horizon MDP M amounts to solving the following optimization problem:

$$\sup_{\pi \in \Pi^{\text{HR}}} V^\pi(s, 1),$$

for all $s \in \mathcal{S}$, where the supremum is taken among all history-dependent randomized policies. This leads us to define the notion of **optimal value function** as follows. The optimal value function in a finite-horizon MDP M is a mapping $V^* : \mathcal{S} \times [H] \rightarrow \mathbb{R}$, such that

$$V^*(s, h) = \max_{\pi \in \Pi^{\text{HR}}} \mathbb{E}^\pi \left[\sum_{t=h}^{H-1} r_t(s_t, a_t) + r_H(s_H) \middle| s_h = s \right]$$

If there exists a policy π^* such that $V^{\pi^*}(s, h) = V^*(s, h)$ for all $s \in \mathcal{S}$ and $h \in [H]$, then π^* is called an **optimal policy** in M . Optimization over the space of all history-dependent randomized policies could be cumbersome, if possible at all. However, as we will see, we can restrict attention only to Markov deterministic policies. In other words, for any discrete finite-horizon MDP, there always exists a Markov deterministic optimal policy.

3 Bellman Equations

3.1 Finding the Value of a Fixed Markov Randomized Policy

The backward induction procedure in the preceding sections provides insights into an efficient method for computing $V^\pi(s, 1)$ for any Markov randomized policy π . More concretely, in the following theorem we formalize that the value function of any Markov randomized policy satisfies the Bellman equation:

Theorem 1 (Bellman's equation for π) *Let $\pi \in \Pi^{\text{MR}}$. Then,*

$$V^\pi(s, H) = r_H(s), \quad s \in \mathcal{S}_{\text{term}}$$

Furthermore, for all $h = H - 1, H - 2, \dots, 1$,

$$V^\pi(s, h) = \mathbb{E}_{a \sim \pi(s, h)} [r_h(s, a)] + \mathbb{E}_{a \sim \pi(s, h)} \left[\sum_{x \in \mathcal{S}} P_h(x|s, a) V^\pi(x, h + 1) \right], \quad s \in \mathcal{S}.$$

For a Markov deterministic policy $\pi \in \Pi^{\text{MD}}$, the second relation above simplifies to:

$$V^\pi(s, h) = r_h(s, \pi(s, h)) + \sum_{x \in \mathcal{S}} P_h(x|s, \pi(s)) V^\pi(s, h+1)$$

for all $s \in \mathcal{S}$ and $h = H-1, \dots, 1$.

Proof. Let $\pi \in \Pi^{\text{MR}}$ and $s \in \mathcal{S}$. For $h \in [H-1]$, we have

$$\begin{aligned} V^\pi(s, h) &= \mathbb{E} \left[\sum_{t=h}^{H-1} r_t(s_t, a_t) + r_H(s_H) \middle| s_h = s, a_t \sim \pi(s_t, t), \forall t \right] \\ &= \mathbb{E}_{a \sim \pi(s, h)} [r_h(s, a)] + \mathbb{E} \left[\sum_{t=h+1}^{H-1} r_t(s_t, a_t) + r_H(s_H) \middle| s_h = s, a_t \sim \pi(s_t, t), \forall t \right] \\ &= \mathbb{E}_{a \sim \pi(s, h)} [r_h(s, a)] \\ &\quad + \sum_{x \in \mathcal{S}} \mathbb{P}(s_{h+1} = x | s_h = s, a_h \sim \pi(s_h, h)) \mathbb{E} \left[\sum_{t=h+1}^{H-1} r_t(s_t, a_t) + r_H(s_H) \middle| s_{h+1} = x, a_t \sim \pi(s_t, t), \forall t \right] \\ &= \mathbb{E}_{a \sim \pi(s, h)} [r_h(s, a)] + \sum_{x \in \mathcal{S}} \mathbb{P}(s_{h+1} = x | s_h = s, a_h \sim \pi(s_h, h)) V^\pi(x, h+1) \\ &= \mathbb{E}_{a \sim \pi(s, h)} [r_h(s, a)] + \mathbb{E}_{a \sim \pi(s, h)} \left[\sum_{x \in \mathcal{S}} P_h(x|s, a) V^\pi(x, h+1) \right]. \end{aligned}$$

□

The above theorem provides a procedure to compute $V^\pi(s, 1)$ for any Markov randomized policy.

3.2 Bellman's Optimality Equation

We now provide two theorems that are fundamental results in the theoretical foundations of finite-horizon MDPs:

Theorem 2 *Suppose that the state space $\mathcal{S}$ is discrete. Then there exists a Markov deterministic policy which is optimal.*

Theorem 2 implies that when seeking an optimal policy in a finite-horizon MDP with a discrete state space, we can restrict our attention to those in Π^{MD} . This further implies that for a discrete $\mathcal{S}$,

$$\sup_{\pi \in \Pi^{\text{HR}}} V^\pi(\cdot, 1) = \sup_{\pi \in \Pi^{\text{MD}}} V^\pi(\cdot, 1) = \max_{\pi \in \Pi^{\text{MD}}} V^\pi(\cdot, 1)$$

Theorem 3 *The optimal value function V^* is the solution to the Bellman optimality equations:*

$$\begin{aligned} V^*(s, H) &= r_H(s) \\ \text{for } h &= H-1, \dots, 1 \\ V^*(s, h) &= \max_{a \in \mathcal{A}_s} \left(r_h(s, a) + \sum_{x \in \mathcal{S}} P_h(x|s, a) V^*(x, h+1) \right) \end{aligned}$$

Equivalently, $\pi \in \Pi^{\text{MD}}$ is optimal if and only if it attains the maximum in the Bellman optimality equations: For all $s \in \mathcal{S}$ and $h \in [H-1]$,

$$\pi^*(s, h) \in \operatorname{argmax}_{a \in \mathcal{A}_s} \left(r_h(s, a) + \sum_{x \in \mathcal{S}} P_h(x|s, a) V^*(x, h+1) \right).$$

Theorem 3 presents a procedure to find both the optimal value and an optimal policy. The procedure is presented in Algorithm 1.

Algorithm 1 Dynamic Programming

Initialize: For all terminal states $s \in \mathcal{S}_{\text{term}}$, $V^*(s, H) = r_H(s)$.

for $h = H - 1, \dots, 1$ **do**

 For all s

$$V^*(s, h) = \max_{a \in \mathcal{A}_s} \left(r_h(s, a) + \sum_{x \in \mathcal{S}} P_h(x|s, a) V^*(x, h+1) \right)$$

$$\pi^*(s, h) \in \operatorname{argmax}_{a \in \mathcal{A}_s} \left(r_h(s, a) + \sum_{x \in \mathcal{S}} P_h(x|s, a) V^*(x, h+1) \right)$$

end for

Output: π^*

Remark 2 You may wonder why a stationary policy may not be optimal in the finite-horizon setting, contrary to the discounted infinite-horizon setting. We clarify this using an example. Consider RiverSwim (Figure 2) with $L = 6$, and let $H = 15$. We can show that in expectation it takes around 14 time steps² to reach s_6 (the high-reward state) from state 1. Now if the agent is in state 1 at $t = 1$, she has the chance to reach state 6 by end of the interaction, and then to collect a reward of 1 there, so that $\pi^*(1, 1) = \text{right}$. However, if, for example, she is at state 1 at any time $t \geq 11$, she surely cannot reach state 6 before the game ends, and hence it is optimal if she instead takes the action **left** and collects a reward of 0.05, hence,

$$\pi^*(1, 11) = \pi^*(1, 12) = \dots = \pi^*(1, 15) = \text{left}.$$

Thus, the choice of the optimal action at state 1 (and other states) depends on time. In other words, a stationary policy is clearly sub-optimal.

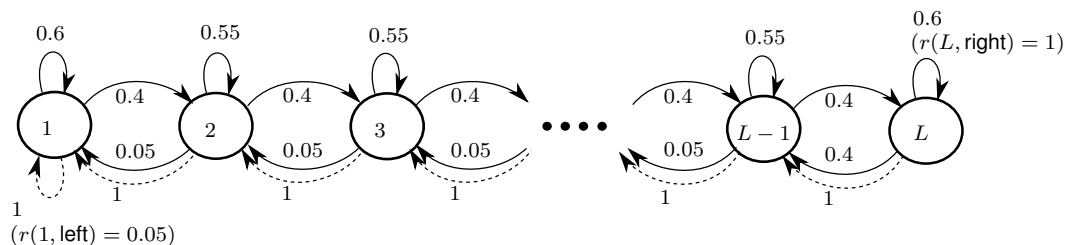


Figure 2: The RiverSwim MDP

4 Examples

Example 1 (Finding the Best Route). We are given a network modeled using the directed graph shown in Figure 4, where the number on each edge represents the weight of that edge. Suppose that we are interested in finding the route between Node 1 and Node 8 with the maximal wight-sum. (For instance, one possible route is $1 \rightarrow 3 \rightarrow 5 \rightarrow 8$ with a weight-sum of $4 + 5 + 1 = 10$.) This problem can be cast as a finite-horizon MDP. The states are nodes in the graph and Node 8 constitutes the only the terminal state, $\mathcal{S} = \{1, 2, \dots, 8\}$ and $\mathcal{S}_{\text{term}} = \{8\}$. At each state (node), available actions are outgoing edges, for example, $\mathcal{A}_1 = \{\text{'go to 2'}, \text{'go to 3'}, \text{'go to 4'}\}$. Rewards are the corresponding edge weights, and the

²This is related to the diameter of RiverSwim; see the lecture on average-reward MDPs.

terminal reward is 0. Finally, the horizon is $H = 3$. Dynamic programming yields

$$h = H = 4 : V^*(8, 4) = 0$$

$$h = 3 : \begin{cases} V^*(5, 3) = 1 + V^*(8, 4) = 1, & \pi^*(5, 3) = \text{'go to 8'} \\ V^*(6, 3) = 2 + V^*(8, 4) = 2, & \pi^*(6, 3) = \text{'go to 8'} \\ V^*(7, 3) = 6 + V^*(8, 4) = 6, & \pi^*(7, 3) = \text{'go to 8'} \end{cases}$$

$$h = 2 : \begin{cases} V^*(2, 2) = \max \{4 + V^*(5, 3), 5 + V^*(6, 3)\} = 7, & \pi^*(2, 2) = \text{'go to 6'} \\ V^*(3, 2) = \max \{5 + V^*(5, 3), 6 + V^*(6, 3), 1 + V^*(7, 3)\} = 8, & \pi^*(3, 2) = \text{'go to 6'} \\ V^*(4, 2) = 2 + V^*(7, 3) = 8, & \pi^*(4, 2) = \text{'go to 7'} \end{cases}$$

$$h = 1 : V^*(1, 1) = \max \{2 + V^*(2, 2), 4 + V^*(3, 2), 6 + V^*(4, 2)\} = 14, \quad \pi^*(1, 1) = \text{'go to 3'}$$

Hence, in view of π^* , the optimal route when starting from Node 1 is $1 \rightarrow 4 \rightarrow 7 \rightarrow 8$, with a weight-sum of $V^*(1, 1) = 14$.

Example 2 [2]. An unscrupulous innkeeper charges a different rate for a room as the day progresses, depending on whether has many or few vacancies. His objective is to maximize his expected total income during the day. Let x be the total number of empty rooms at the start of the day, and let y be the number of customers that will ask for a room in the course of the day. We assume (somewhat unrealistically) that the innkeeper knows y with certainty, and upon arrival of a customer, quotes one of m prices $r_i, i = 1, \dots, m$, where $0 < r_1 \leq r_2 \leq \dots \leq r_m$. A quote of a rate r_i is accepted with probability p_i and is rejected with probability $1 - p_i$, in which case the customer departs, never to return during that day.

(i) Show that the maximal expected income as a function of x and y satisfies:

$$V(x, y) = \max_{i=1, \dots, m} [p_i(r_i + V(x-1, y-1)) + (1-p_i)V(x, y-1)],$$

for all $x \geq 1$ and $y \geq 1$ with $V(x, 0) = V(0, y) = 0$ for all x and y .

(ii) Show that $V(x-1, y) \leq V(x, y)$ for all $x \geq 1$ and $y \geq 0$.

(iii) Assume that

$$\begin{aligned} p_1 r_1 &\leq p_2 r_2 \leq \dots \leq p_m r_m \\ p_1 &\geq p_2 \geq \dots \geq p_m. \end{aligned}$$

Show that the innkeeper should always charge the highest rate r_m .

Solution. (Part (i).) Let states be equal to the number of free rooms $S = \{0, 1, \dots\}$. As state $x \geq 1$, the innkeeper has $A_x = \{1, 2, \dots, m\}$, where action i corresponds to quoting a rate r_i . We further have $A_0 = \emptyset$.

Transition probabilities are given below:

$$p(x-1|x, i) = p_i, \quad p(x|x, i) = 1 - p_i, \quad \forall x \geq 1.$$

Furthermore, the innkeeper's reward function is: $r(x, i) = p_i r_i$ for all i and $x \geq 1$, and $r(0, \cdot) = 0$. It is assumed that p_i is monotonically nondecreasing in i .

Dynamic programming for this problem yields:

$$\begin{aligned} V(x, 0) &= 0, \quad \forall x \geq 0, \\ V(x, y) &= \max_{i=1, \dots, m} [p_i(r_i + V(x-1, y-1)) + (1-p_i)V(x, y-1)], \quad \forall x \geq 0, \\ V(0, y) &= 0, \quad \forall y. \end{aligned}$$

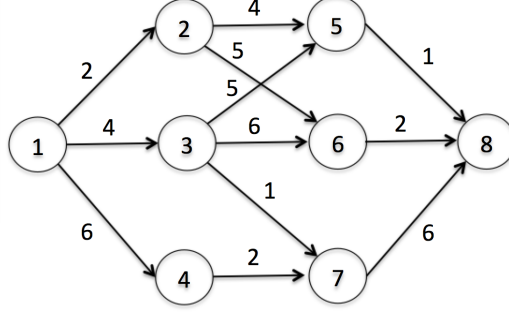


Figure 3: The Network for Example 2

Part (ii). We prove by induction on y that $V(x, y) \geq V(x - 1, y)$ for all $x \geq 1$ and all y . Indeed this is true for $y = 0$. Assuming that this is true for a given y , we will prove that

$$V(x, y + 1) \geq V(x - 1, y + 1), \quad \forall x \geq 1.$$

This relation holds for $x = 1$ since $r_i > 0$. For $x \geq 2$, using the dynamic programming recursion, this relation is written as:

$$\begin{aligned} & \max_{i=1, \dots, m} [p_i(r_i + V(x - 1, y)) + (1 - p_i)V(x, y)] \\ & \geq \max_{i=1, \dots, m} [p_i(r_i + V(x - 2, y)) + (1 - p_i)V(x - 1, y)]. \end{aligned}$$

By the induction hypothesis (i.e., $V(x, y) \geq V(x - 1, y)$ for all x), each of the terms on the left-hand side is no less than the corresponding term on the right-hand side, and so the above relation holds.

Part (iii). The optimal rate is the one that maximizes in the dynamic programming, or equivalently the maximizer of

$$p_i r_i - p_i(V(x, y - 1) - V(x - 1, y - 1)).$$

The highest rate r_m simultaneously maximizes $p_i r_i$ and minimizes p_i . Since $V(x - 1, y - 1) \leq V(x, y - 1) \leq 0$ (as proved above), we see that the highest rate maximizes the above relation.

Example 3 (Matrix Chain Multiplication). Suppose you are given a sequence of matrix multiplications

$$M_1 M_2 \dots M_k M_{k+1} \dots M_N,$$

with each matrix M_k has dimension $n_k \times n_{k+1}$. The order in which multiplications are carried out can make a difference. For example, if $n_1 = 1$, $n_2 = 10$, $n_3 = 1$, and $n_4 = 10$, the calculation $((M_1 M_2) M_3)$ requires 20 scalar multiplications, whereas $(M_1 (M_2 M_3))$ requires 200 scalar multiplications.

Suppose we are interested in finding the optimal multiplication order, i.e., the one leading to the least computation. This can be done by casting the problem as a finite-horizon MDP. The state space of the corresponding MDP is

$$\mathcal{S} = \{(1, 1), (1, 2), \dots, (1, N), (2, 2), (2, 3), \dots, (2, N), (3, 3), \dots, (N, N)\}.$$

At each state (i, j) , we have $\mathcal{A}_{(i, j)} = \{i, i + 1, \dots, j - 1\}$. For each state $s = (i, j) \in \mathcal{S}$, we let $V^*(i, j)$ be the minimum number of multiplications needed to compute $M_i M_{i+1} \dots M_j$. Then, using dynamic programming, we have the following recursion:

$$\begin{aligned} V^*(i, j) &= \min_{i \leq k < j} (V^*(i, k) + V^*(k + 1, j) + n_{i-1} n_k n_j), \quad i < j \\ V^*(i, i) &= 0. \end{aligned}$$

This recursion straightforwardly follows from the fact that $A_i A_{i+1} \dots A_j = (A_i \dots A_k)(A_{k+1} \dots A_j)$. The minimal multiplication cost will be $V^(1, N)$, which can be computed recursively. To recover the optimal chain for multiplication, it would be enough to save at each step the index k which minimizes the relation for $V^*(i, j)$. For example, for $N = 3$, $n_1 = 2$, $n_2 = 10$, and $n_4 = 1$, the optimal ordering is found to be $M_1(M_2 M_3)$.*

References

- [1] Martin L Puterman. *Markov decision processes: Discrete stochastic dynamic programming*. John Wiley & Sons, 2005.
- [2] Dimitri Bertsekas. *Dynamic programming and optimal control: Volume I*, volume 1. Athena scientific, 2012.